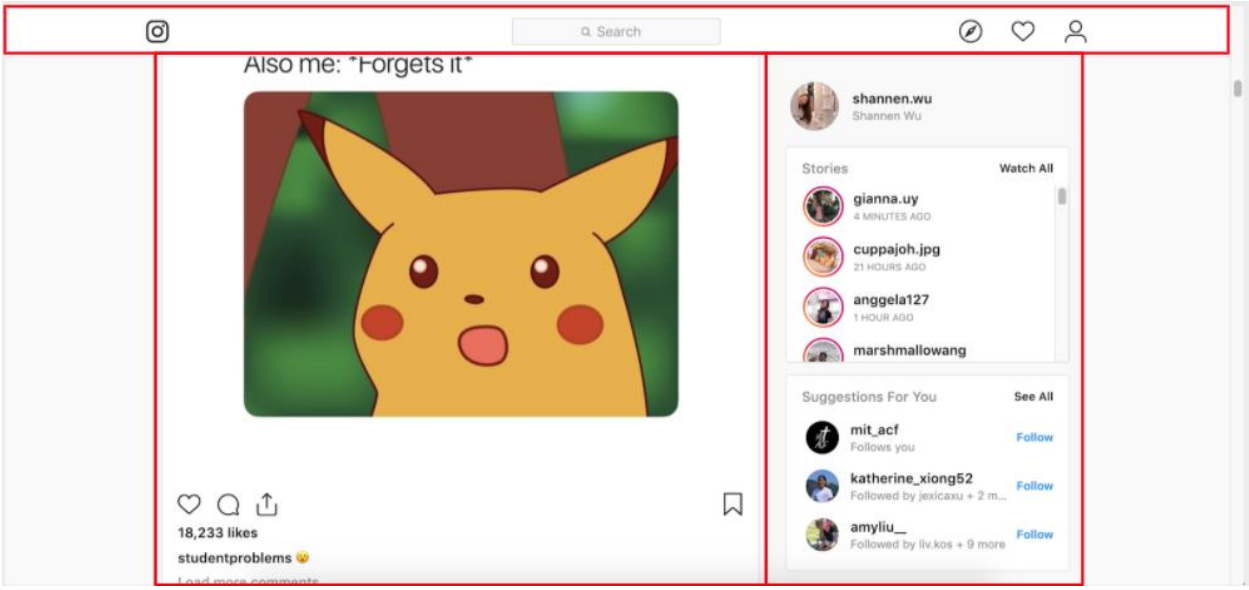
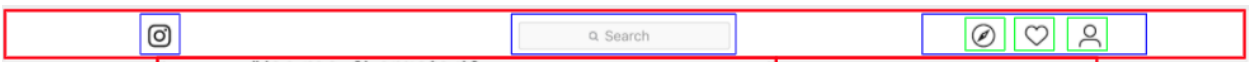
React Overview.. the FRONT-END!

React is a **front end, JavaScript library** for building components.

You will make your app out of these **components,** which are basically “boxes” that show up in your website**.** Think of React as a **way of organizing your front end.** Your whole app is a component, the <App /> component, and is split up into components, which are split up, etc etc.



React components can have **children** and **parents**. If we zoom in on the React component (or box) that is the navbar at the top of the instagram site, we can see this in action.



Here, the red box is a parent component. It has three blue child components. The rightmost child component is also the parent component of three more green child components.

If you want to include **anything in your website** (like a button, chat window, social media post, subpage, login page, etc.) your instinct should be **“let’s make a React component for that!”**

Example.js: The "Example" component would be defined in *"Example.js"* which is shown below with comments detailing the major aspects of React.

```
import React, { useState } from "react";
import "./Example.css";
import ExampleChildComponent from "./ExampleChildComponent.js"
const Example = (props) => {
 const [exampleState1, setExampleState1] = useState("dummy message")
 const [exampleState2, setExampleState2] = useState(0)

 useEffect(() => {
      get("/api/anEndpoint", {param1: "value"}).then((serverResponse) => {
            setExampleState1(serverResponse)
      })
 }, [])

 const addOneToState2 = () => {
   setExampleState2(exampleState2 + 1)
 }

 return (
     <div>
       <h1 className="headerStyle1">{props.title}</h1>
```

```
        <ExampleChildComponent
          exampleProp1={10}
          exampleProp2={exampleState2}
        />
        <button onClick={addOneToState2} />
      </div>
  );

}
export default Example;
```

React Topics

Components: A component is essentially your own custom HTML tag that you define, and can take in props as input. You use a component by typing `<MyComponent myprop1="this is a string" myprop2={25} />` (here, `props.myprop1` is "this is a string" and `props.myprop2` is 25).

Your whole app will be in a 'component' called App (so your website code will be literally just `<App />`), but you'll divide it into multiple components, which themselves will be made up of components, etc etc. Each component will be defined in its own javascript file.

Props: We usually want our components to have inputs! Props are our way of specifying inputs to our components. Props are read-only pieces of information, such as a variable value or function, that a child component takes as inputs from the parent component. In the MyComponent example above, MyComponent can access the props `props.myprop1` and `props.myprop2`, which were passed in from MyComponent's parent.

State: Each component has an internal state, which is a bunch of variables that represent information about the component's current situation. Variables that are part of the state are declared through:
`const [variable1, setVariable1] = useState(initialValueOfVariable1)`
where `setVariable1` is a function that can be used to set the value of `variable1`.

Router: A nice way of organizing your website's internal structure by having different pathNames route to different React components (remember React components can represent entire pages in your website!) So if we wanted

- Feed component: https://examplewebsite.com/
- Profile component: https://examplewebsite.com/profile
- NotFound component: https://examplewebsite.com/anyOtherPath

```
<Router>
  <Feed path="/" />
  <Profile path="/profile/" />
  <NotFound default />
</Router>
```

Lifecycle/useEffect: Normally, whenever the props or state changes, React has to re-render the component onto our screen.

For example if for some reason `props.title` changes then React would need to update that and re-return the html code with a different `props.title`.

So, what React will do is just run all of the component code again. So, if we just did some stuff (e.g. get data from some source) right before the return, it would get the data both once at the beginning and then get the data again once everytime the props or state changes...

Sometimes we want to do stuff just once, at the beginning, and not **every time**. Or sometimes we want to do stuff once at the beginning and every

time a specific prop changes. For these use cases we use the 'useEffect' hook.

The syntax of useEffect is:

`useEffect(someFunction, [])`: runs once at the beginning

`useEffect(someFunction, [props.title])`: runs once at the beginning and every time props.title changes

`useEffect(someFunction, [props.title, exampleState2])`: runs once at the beginning and every time props.title changes **or** exampleState2 changes

Normally, we will mainly use `useEffect` for any data retrieval we need to do once at the beginning.

What is .then()?:

(note: this is a general javascript thing, not a React thing)
This will be covered in a lot more depth next week. Some functions in javascript, like 'get' and 'post', are **asynchronous**. So, if I ran

```
get("/api/cats")
console.log("hello")
```

then the get function, being asynchronous, would start running **in the background**, and 'hello' would be printed **before** the get request finished.

If we wanted to do something like print the result of the get request, we would need to wait for the get request to finish before printing the result. To wait, we use the .then function:

```
get("/api/cats").then((response) => {
    console.log(response)
})
```

and what this does is calls the 'get' function, and **then** once the function finishes, it takes the response and prints it.

'*asynchronousfunction*.then(*secondfunction*)' means that asynchronous function runs, and **then** after it finishes, *secondfunction* runs.

FAQ

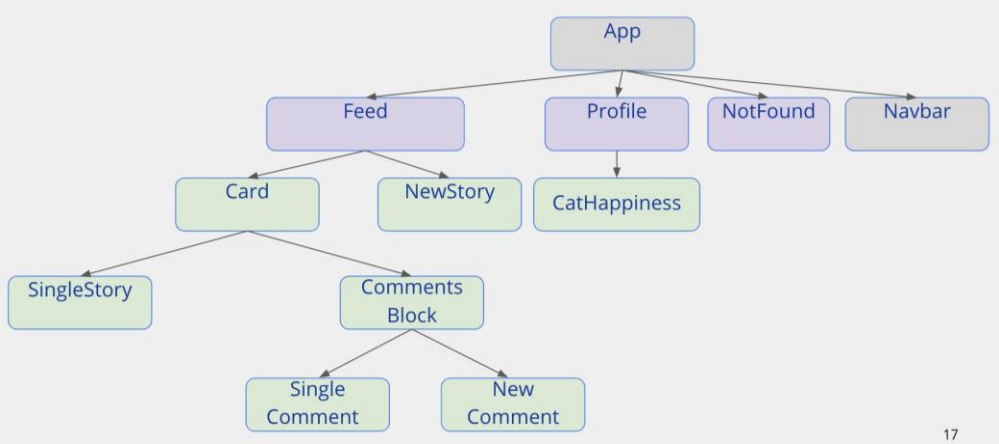
What are states? What's the difference between states and props?

React states are ways of storing information that might be relevant to the appearance/function of a React component.

React props (short for properties) also store information. However, **props** are **read-only** pieces of information **passed down from the parent** component, whereas **states** are **updateable** and **specific to a component**. You can think of props as the "unchangeable settings" of your component (that you may want to specify from the parent), whereas the state contains the "updateable settings."

There are so many files. How do we keep track of all of them?

Keeping a component tree in mind helps a lot with seeing how each file fits in with the big picture.



Man. This is still super confusing.

1. Come to office hours / ask on Piazza (experience our 1 minute response time) / feel free to come up to one of us and have us explain things one on one!

2. Look at the workshops (make sure you have catbook-react cloned, and then

- close out of any unsaved files
- git fetch
- git reset --hard
- git checkout wX-complete

(where X is the workshop number. For a full frontend example check out w3-complete). If you have any setup troubles ask on piazza, or if any part of the workshop is confusing, ask on piazza!

3. Please suggest other things that can be explained better / added to this guide!